

Oltre il modello DB singolo + business logic + interfaccia

Polyglot persistence, eventi, viste specializzate e architetture flessibili

NoSQL

Materiale aggiuntivo per blocchi 13-16

1. Il modello semplice

Il modello “un database, una business logic, una interfaccia” è ottimo per partire, ma diventa stretto quando crescono volume, domini e access pattern.

Il modello iniziale

Componente	Responsabilità
Interfaccia	mostra schermate e raccoglie input
Business logic	valida regole e coordina operazioni
Database singolo	conserva tutto: transazioni, report, ricerca, log

È un modello didatticamente utile: è facile da spiegare, testare e distribuire.

Quando funziona bene

- ▶ dominio piccolo o medio;
- ▶ team unico;
- ▶ carico prevedibile;
- ▶ query non troppo eterogenee;
- ▶ dati principalmente tabellari;
- ▶ requisiti di disponibilità gestibili con replica e backup ordinari.

Perché diventa limitante

Segnale	Conseguenza
molti casi d'uso diversi scritture ad alto volume ricerca testuale complessa relazioni profonde dashboard quasi real-time team multipli	un solo schema deve servire troppe domande report e transazioni si disturbano SQL diventa artificiale o lento molte join e query difficili carico analitico sul DB operativo deploy e ownership diventano accoppiati

Un piccolo ufficio può usare un unico armadio per contratti, fatture, manuali, badge, mappe e log.

Quando l'organizzazione cresce, l'armadio unico non basta più:

- ▶ i contratti richiedono controllo formale;
- ▶ i manuali devono essere ricercabili;
- ▶ le mappe devono mostrare relazioni;
- ▶ i log devono essere analizzati velocemente;
- ▶ le dashboard devono leggere senza bloccare il lavoro operativo.

2. Paradigmi più generali

A scala maggiore non si sceglie solo un database: si progetta un flusso di dati tra componenti con responsabilità diverse.

Definizione operativa

Usare più tecnologie di persistenza nello stesso sistema, ciascuna scelta per un modello dati o access pattern specifico.

- ▶ Relazionale per dati core e vincoli.
- ▶ Documentale per aggregate flessibili.
- ▶ Search per testo e log.
- ▶ Graph per relazioni e dipendenze.
- ▶ Cache/key-value per letture frequenti e stato volatile.
- ▶ Time-series per metriche e segnali temporali.

Idea

I componenti non si parlano solo con chiamate dirette. Pubblicano eventi: “ticket creato”, “asset aggiornato”, “ordine spedito”, “manuale indicizzato”.

Gli eventi permettono a più consumatori di costruire viste diverse senza bloccare il flusso principale.

Parte	Scopo
Command model	scrivere correttamente, validare regole, proteggere invarianti
Read model	leggere velocemente in una forma vicina alla schermata o dashboard
Event stream	propagare cambiamenti tra modelli
Projection	trasformare eventi in viste consultabili

Schema concettuale

Livello	Responsabilità	Esempio
API/service	regole e casi d'uso	ticket, interventi
DB core	fonte autorevole	PostgreSQL
Event bus	distribuzione eventi	Kafka/queue
Read model	query specializzate	search, graph, cache
Analytics	storico e BI	warehouse/lake
Dashboard	decisione operativa	Metabase/Kibana

- ▶ più flessibilità, ma più componenti da osservare;
- ▶ query più veloci, ma dati duplicati;
- ▶ servizi meno accoppiati, ma consistenza spesso eventuale;
- ▶ ogni store è più adatto al suo compito, ma aumenta il costo operativo;
- ▶ il modello mentale passa da “tabella unica” a “flusso di eventi e viste”.

3. Caso oltre la telemetria

Manteniamo il laboratorio telemetria, ma aggiungiamo un esempio diverso: gestione guasti e interventi sul campo in una utility energetica.

Caso: gestione guasti e interventi

Scenario informale

Dopo un evento meteo, arrivano segnalazioni di guasto da clienti, tecnici, sistemi interni e canali partner. L'azienda deve capire priorità, asset coinvolti, squadre disponibili, documentazione tecnica e impatto sui clienti.

Non è solo telemetria: è coordinamento operativo, documenti, asset, relazioni, ricerca e workflow.

Domande da soddisfare

- ❶ Quali segnalazioni sono aperte e con quale priorità?
- ❷ Quali asset sono coinvolti e quali dipendenze hanno?
- ❸ Quali clienti o zone sono potenzialmente impattati?
- ❹ Quali manuali o interventi passati sono rilevanti?
- ❺ Quale squadra è più adatta e disponibile?
- ❻ Quali KPI operativi deve vedere il responsabile?

Un solo database: che cosa succede

Esigenza	Problema se tutto resta nello stesso DB
workflow ticket	gestibile, ma cresce la complessità
documenti tecnici	ricerca testuale debole o costosa
topologia asset	join ricorsive e query difficili
foto e allegati	storage e backup poco adatti
dashboard operative	carico analitico sul DB transazionale
suggerimenti interventi simili	servono ranking e similarità

Architettura proposta

Componente	Tecnologia candidata	Responsabilità
Ticket core	PostgreSQL	stato ufficiale, SLA, vincoli
Work order form	Document DB	moduli tecnici variabili
Asset topology	Graph DB	dipendenze rete e impatto
Manuali e note	Search engine	full-text e ranking
Sessioni/lock	Key-value	stato temporaneo operativo
Event stream	broker/queue	propagazione cambiamenti
Dashboard	BI/Search	KPI e drill-down

Regola

Duplicare dati non è vietato, ma bisogna dichiarare quale copia è autorevole e quale è una vista derivata.

- ▶ Stato del ticket: autorevole in PostgreSQL.
- ▶ Documento work order: autorevole nel document store.
- ▶ Indice di ricerca: copia derivata.
- ▶ Grafo asset: può essere derivato da sistemi asset management.
- ▶ Dashboard: vista derivata e potenzialmente ritardata.

- ① Il customer care crea un ticket guasto.
- ② Il servizio ticket salva lo stato ufficiale.
- ③ Viene pubblicato l'evento `ticket.created`.
- ④ Un consumer aggiorna il motore search.
- ⑤ Un consumer collega ticket e asset nel grafo.
- ⑥ Un consumer aggiorna la dashboard operativa.
- ⑦ Un servizio field crea o aggiorna il work order.

Esempio evento

```
{  
  "event_type": "ticket.created",  
  "ticket_id": "T-2026-90012",  
  "opened_at": "2026-05-14T09:20:00Z",  
  "category": "outage",  
  "region": "center",  
  "asset_hint": "CAB-1821",  
  "priority": "urgent"  
}
```

Proiezioni diverse dello stesso evento

Consumer	Trasformazione	Store
search-indexer	indicizza testo e metadati	search
asset-linker	collega ticket ad asset e zone	graph
dashboard-projector	aggiorna KPI per regione	read model
notification-service	invia alert a squadra	queue/cache
workorder-service	crea modulo intervento	document DB

Query che diventano naturali

- ▶ PostgreSQL: “quanti ticket urgenti aperti abbiamo per SLA?”.
- ▶ Search: “trova casi simili con descrizione 'cabina allagata'”.
- ▶ Graph: “quali clienti dipendono dagli asset connessi a CAB-1821?”.
- ▶ Document DB: “mostra il modulo tecnico completo dell'intervento”.
- ▶ Cache: “quale squadra ha lock operativo sul ticket?”.

4. Confronto con telemetria

Il laboratorio telemetria resta valido: è un caso specifico. Il nuovo esempio mostra perché, in sistemi grandi, più modelli convivono.

Telemetria vs gestione guasti

Aspetto	Telemetria	Gestione guasti
Dato principale	lettura sensore	ticket, asset, work order
Forma	evento tecnico semi-strutturato	workflow multi-oggetto
Query	trend, KPI, alert	stato, ricerca, impatto, assegnazione
Store centrale lab	MongoDB	più store specializzati
Coerenza	dashboard può tollerare ritardi	alcune azioni richiedono forte controllo

Perché non buttare via il relazionale

Il punto non è sostituire PostgreSQL con NoSQL.

- ▶ Ticket, SLA e stati ufficiali hanno vincoli e transazioni.
- ▶ Contratti, fatture e anagrafiche core restano spesso relazionali.
- ▶ NoSQL entra dove il modello relazionale diventa innaturale o troppo costoso per uno specifico access pattern.
- ▶ La buona architettura combina modelli, non segue mode tecnologiche.

Domande architetturali da fare

- 1 Qual è il sistema of record?
- 2 Quali viste sono derivate?
- 3 Quanto ritardo è accettabile tra scrittura e lettura?
- 4 Come ricostruisco una vista se si corrompe?
- 5 Come traccio quale evento ha aggiornato quale store?
- 6 Come faccio rollback o compensazione?

5. Costi e rischi

Più flessibilità non è gratis: serve disciplina su contratti, eventi, osservabilità e ownership.

- ▶ Duplicazione incontrollata.
- ▶ Eventi senza schema e senza versionamento.
- ▶ Consistenza eventuale non spiegata agli utenti.
- ▶ Debug difficile: il dato passa da molti componenti.
- ▶ Team senza ownership chiara.
- ▶ Costi cloud e operativi non monitorati.

Problema	Pratica
eventi incompatibili	schema registry o contratti versionati
duplicazione	dichiarare source of truth e viste derivate
debug difficile	correlation id e tracing
ritardi tra store	metriche di lag e retry idempotenti
ricostruzione viste	replay eventi o job di backfill
ownership confusa	responsabilità per servizio e store

Regola conservativa

Partire semplice è corretto. Evolvere è necessario quando un requisito reale rende il modello semplice troppo costoso o fragile.

Segnali di evoluzione:

- ▶ query molto diverse tra loro;
- ▶ carichi di scrittura e lettura in conflitto;
- ▶ team o domini con cicli di vita separati;
- ▶ requisiti di ricerca, grafo, streaming o analytics non naturali nel DB core.

Dato lo scenario gestione guasti, dividere la classe in gruppi e chiedere:

- 1 quali dati sono system of record?
- 2 quali dati sono viste derivate?
- 3 quali store usereste e perché?
- 4 quale consistenza serve per ogni lettura?
- 5 quale componente può fallire senza bloccare tutto?
- 6 quale metrica usereste per capire se l'architettura funziona?

- ▶ Il modello a tre livelli è un ottimo punto di partenza.
- ▶ A scala alta compaiono access pattern incompatibili con un solo database.
- ▶ L'architettura moderna separa source of truth, eventi e read model.
- ▶ NoSQL è utile quando è collegato a un modello architetturale, non quando è scelto isolatamente.
- ▶ Il laboratorio telemetria è un tassello: il ragionamento generale è più ampio.